

SAT Theory Lean 4 Framework Snippet

```

/-
SAT Theory
4D Tensorial + Discrete Twist () Integration
Lean 4 Framework
-/

import Mathlib.Data.Real.Basic
import Mathlib.Analysis.Calculus.Deriv
-- Tensor libraries and integration helpers would go here

-- I. Numeric parameters and physical constants
structure SATParams :=
  (T A _f g M2 c c _c k a : )

-- II. Topological invariants (discrete sector)
structure TopoInv :=
  (L_wind L_link W_writhe : )

def Q ( : TopoInv) : := .L_wind + .L_link + .W_writhe
def Q_stable ( : TopoInv) : Prop := Q 3

-- III. Mass Spectrum
def m_topo (m : ) ( : TopoInv) : := m / (Q : )
def m_ind (params : SATParams) (E_overlap : ) : := params.g2 / params.M2 * E_overlap
def m_eff (m : ) ( : TopoInv) (params : SATParams) (E_overlap : ) : :=
  m_topo m + m_ind params E_overlap
def m_squared ( : TopoInv) (N : ) (params : SATParams) : :=
  params.k * (N + params. * (Q )2 - params.a)

-- IV. 4D Fields
structure SATFields :=
  ( : ) -- scalar (x~)
  (u : ) -- vector u~(x~)
  ( : ) -- emergent spinor (x~)
  (J : ) -- filament current J~{x~}
  ( : TopoInv) -- discrete twist charge

-- V. Covariant Derivatives (symbolic)
def (f : ) ( : ) : := f -- placeholder
def D (f : ) ( : ) : := f -- gauge covariant

-- VI. Strain and Stress
def S (u : ) ( : ) (x : ) : :=
  u x + u x

def T (u J : _) ( : ) (x : ) : :=
  (0 x)2 + (u 0 x)2 + (J 0 0 x)2

-- VII. EulerLagrange Equations
def EL_ (fields : SATFields) (x : ) : :=
  ( fields. ) x + 3 * Real.sin (3 * fields. x)

```

```

def EL_u (fields : SATFields) ( : ) (x : ) ( : ) : :=
  ( fields.u ) x + * fields.u x + fields. x

def EL_J (fields : SATFields) ( : ) (x : ) : :=
  (fields.J ) x

-- VIII. Discrete Twist Charge Operators
def _add ( : TopoInv) : TopoInv :=
  { L_wind := .L_wind + .L_wind,
    L_link := .L_link + .L_link,
    W_writhe := .W_writhe + .W_writhe }

def _mod3 ( : TopoInv) : TopoInv :=
  { L_wind := .L_wind % 3,
    L_link := .L_link % 3,
    W_writhe := .W_writhe % 3 }

-- IX. BF Topological Action
def S_BF (B A : ) ( : TopoInv) : :=
  x in 0..1, (B 0 x) * ( A 0 x) + (Q : ) -- topological coupling via

-- X. Filament Kernel Contribution (suppresses topological mass)
def filament_kernel (J : ) ( : TopoInv) : :=
  x in 0..1, J 0 0 x * Real.exp (- (Q : ))

-- XI. Total Action (continuous + discrete)
def L_total (fields : SATFields) (params : SATParams) : :=
  fun x =>
    (fields. x)^2 + (fields.u x)^2 + (fields.J x)^2
    + S_BF fields.u fields. fields.
    + filament_kernel fields.J fields.

def S_total (fields : SATFields) (params : SATParams) : :=
  x in 0..1, L_total fields params 0 x

-- XII. Canonical Quantization (symbolic)
structure CanonicalQuantization :=
  ( : )
  ( : )

def canonical_commutator (cq : CanonicalQuantization) ( ' : ) : :=
  if = ' then Real.pi else 0

-- XIII. Example instantiation
def example_params : SATParams :=
{ T := 1.0, A := 1.0, _f := 1.0, g := 0.01, M^2 := 10.0,
  c := 1.0, c := 1.0, _c := 0.1, k := 1.0, := 0.1, a := 0.01 }

def example_topo : TopoInv := {L_wind := 1, L_link := 1, W_writhe := 1}

def example_fields : SATFields :=
{ := fun x => Real.sin x,
  u := fun x => x,
  := fun x => 0,

```

```
J := fun x => 0.1 * x,  
      := example_topo }  
  
#eval m_eff 10.0 example_topo example_params 0.001  
#eval S_total example_fields example_params
```